

TrustHome PH — Versioning & Release Plan (1-Month Timeline)

Compressed from a 12-week plan to **4 weeks**, since the team is AI-assisted. One version ships per week, and each `Scrum_Weekly_Report_Template.docx` submission lines up exactly with one version below — no more mapping sprints to versions, they're now the same thing.

Why this compression works without cutting scope: AI-assisted development mostly removes *typing time* — boilerplate CRUD, standard auth flows, standard database calls. It does not remove *review, integration, and decision time* — someone still has to check the AI's output is correct, wire modules together, and decide on edge cases. So the plan below **keeps every module from the original 12-week plan**, but restructures the order so people work in parallel more aggressively instead of waiting on each other — that's where the real time savings come from, not from cutting features.

Stack (confirmed): React (web) + Firebase (Firestore, Auth, Cloud Functions, Hosting) + Cloudinary for file storage, with the Python bank scraper writing into Firestore via `firebase-admin`. Renters and homeowners are merged into a single account type — every user can browse, book, and list a property, with `admin` as the only separate role. User pages are mobile-portrait; admin pages are desktop-landscape. See the System Build Plan (Firebase Edition) and Design Guide for the full technical and visual spec this plan assumes.

How this maps to your official submissions: Week 1's module list is your "30% Implemented System" for Part B of `SoftDev-Template-Midterm.docx`. The full 14-row inventory is still the same — only the version/week assignment changed.

1. Version Overview

Version	Week	Focus	Milestone
v0.1	Week 1	Foundations	30% Midterm Milestone
v0.2	Week 2	Trust Engine	Core differentiator complete
v0.3	Week 3	Accountability & Integration	All major features complete
v1.0	Week 4	Final Release	Full QA + defense-ready

2. Full Feature Inventory (Part A draft — unchanged from before)

No.	Module / Feature	Priority	Size	Version	Owner
1	Authentication & Accounts	High	M	v0.1	Andresa
2	Listing Management	High	L	v0.1	Carillo
3	Verified Listing Intake & Review	High	M	v0.1	Carillo
4	Search & Browse	High	M	v0.1	Laurente
5	Bank-Acquired Catalog + Scraper	Medium	L	v0.1	Solpico
6	Trust & Fairness Score (schema + formula)	High	L	v0.1 → v0.3	Prades
7	Booking Calendar	High	L	v0.2	Laurente
8	Admin Dashboard & User Management	Medium	M	v0.2	Andresa
9	My Dashboard	Low	S	v0.2	Carillo
10	Loan / Amortization Calculator	Medium	S	v0.2	Solpico
11	Ratings & Reviews	Medium	S	v0.2 → v0.3	Prades
12	Dispute &	High	M	v0.3	Laurente

	Accountability Trail				
13	Public Profile	Low	S	v0.3	Carillo
14	Saved Searches / Alerts	Low	S	v0.3	Andresa

What changed vs. the 12-week plan: modules didn't disappear — some now start earlier as scaffolding (row 6, 11) and finish once their dependency lands, and the Bank Scraper (row 5) moved up to Week 1 since it's fully independent and has zero reason to wait.

3. Week-by-Week Breakdown

v0.1 — Foundations (Week 1) — 30% Midterm Milestone

Goal: a working, demoable core — register, list a verified property, search, and browse real bank listings.

Day 1, before any module work starts: whoever is fastest to set it up (suggest Andresa, since Auth depends on it first) creates the Firebase project (Firestore + Authentication) and the Cloudinary account, then shares the config/API keys with the team — see the [Firebase Setup Plan](#) for exact steps. This is a 30-45 minute task that everything else in Week 1 blocks on, so it should happen first, not whenever it comes up.

Module	Owner	Notes
Authentication & Accounts	Andresa	Sets up the Firebase project and Cloudinary account first, then Firebase Auth. No role picker needed — every account can browse, book, and list. Blocks everyone else — prioritize Day 1-2.
Listing Management	Carillo	Needs Auth; coordinate with Andresa early on the <code>listings</code> collection shape (see System Build Plan §2). Any signed-in user can post a listing.
Verified Listing Intake & Review	Carillo	Same module family as above, build together. Uses Cloudinary for the uploaded ID/ownership doc.
Search & Browse (basic)	Laurente	Can start against a stub/seed <code>listings</code> collection before Carillo's is fully done.
Bank-Acquired Catalog + Scraper	Solpico	Fully independent — start immediately. Update the existing Python scraper's <code>database.py</code> to write to Firestore via <code>firebase-admin</code> instead of SQLite (see System Build Plan §7 for the updated code).
Trust & Fairness Score — schema + formula only	Prades	Build and unit-test the scoring formula (plain JS functions) against seed data now (System Build Plan §5); don't wait for real bookings to exist.

Why Trust & Fairness Score starts this early: the formula itself doesn't need real data to be *built and tested* — only to be *populated*. Getting the schema and calculation logic done in Week 1 means Week 2's Booking Calendar can plug straight into it instead of Prades starting from zero.

v0.2 — Trust Engine (Week 2)

Goal: the two features that prove this isn't a CRUD app.

Module	Owner	Notes
Booking Calendar	Laurente	The core differentiator — conflict-check logic (done in app code against Firestore, not a database query — see System Build Plan §4), status workflow. Top priority this week.
Admin Dashboard & User Management	Andresa	Independent of Booking; build in parallel. Desktop-landscape layout per the Design Guide.
My Dashboard	Carillo	Independent; build in parallel.

Loan / Amortization Calculator	Solpico	Small, self-contained — quick win alongside finishing Bank Catalog polish.
Ratings & Reviews	Prades	Wire up against Laurente's real bookings collection as soon as Completed status exists — coordinate mid-week.

v0.3 — Accountability & Integration (Week 3)

Goal: close the “what stops a scam” gap, and get the Trust Score running on real data end-to-end.

Module	Owner	Notes
Dispute & Accountability Trail	Laurente	Builds directly on Booking Calendar from last week.
Trust & Fairness Score — full integration	Prades	Connect the Week 1 formula to real completed bookings + ratings; verify ranked search actually reorders correctly.
Public Profile	Carillo	Needs Prades's trust score and Laurente's dispute flag — schedule this for the back half of the week.
Saved Searches / Alerts	Andresa	Independent; can be done anytime this week.

v1.0 — Final Release (Week 4)

Goal: everything integrated, tested, and demo-ready.

Day(s)	Focus	Who
Day 1-2	Full regression pass — walk through every module against Test_Plan.xlsx / Test_Case_Template.docx	Everyone, paired
Day 3	Bug fixing from the regression pass	Whoever owns the affected module
Day 4	Seed realistic demo data (per the System Build Plan §10 checklist) + UI polish pass against the Design Guide	Everyone
Day 5	Full dry-run demo + defense Q&A rehearsal	Everyone

4. Per-Member Weekly View

Member	Week 1	Week 2	Week 3	Week 4
Andresa (Scrum Master)	Auth & Accounts	Admin Dashboard & User Mgmt	Saved Searches / Alerts	QA + demo prep
Carillo	Listing Mgmt + Verified Intake	My Dashboard	Public Profile	QA + demo prep
Laurente	Search & Browse	Booking Calendar	Dispute & Accountability Trail	QA + demo prep
Prades	Trust Score (schema + formula)	Ratings & Reviews	Trust Score (full integration)	QA + demo prep
Solpico	Bank Scraper + Catalog	Loan Calculator	Support / buffer	QA + demo prep

Solpico has lighter load in Week 3 by design — that's your buffer slot if any other module runs long, since something almost always does.

5. What “using AI to work fast” changes in practice

- **Use AI for:** boilerplate CRUD (auth forms, listing forms), standard Firestore queries, first-draft React components against the Design Guide's spec, generating seed/sample data, writing test cases from the logic already

documented in the System Build Plan.

- **Still do yourselves:** reviewing AI-generated code before merging (especially the overlap-check logic and trust score formula — get these exactly right, they're what the panel will ask about), deciding on edge cases the AI won't know about (e.g., what happens when two disputes hit the same booking), setting up **Firestore Security Rules** properly (AI can draft these but someone must actually verify one user can't read another user's private data), and the actual integration between each member's module.
- **Daily, not weekly, syncs are worth it at this pace.** A 10-minute daily check-in ("what did you finish, what are you touching today, any blockers") prevents two people from editing the same schema differently — a bigger risk now that everyone moves faster.